Proyecto Fundamentos Básicos de Python - Programación 4

Nombre: Sara López Quintero
Docente: Alejandro Rodas Vásquez
Universidad Tecnológica de Pereira

1. Solución implementada

- **api/datos.py**: lee el archivo con `pandas` y convierte cada fila en un diccionario. Filtra los registros por departamento, municipio y cultivo sin distinguir mayúsculas ni tildes, y se detiene al alcanzar el número de registros pedido. Calcula la mediana de pH, Fósforo (P) y Potasio (K) ordenando los valores y tomando el central, o el promedio de los dos centrales si la cantidad es par. Los valores no numéricos (por ejemplo `<0,14` o `ND`) se descartan del cálculo.
- **ui/interfaz.py**: solicita los cuatro datos de entrada, valida que no estén vacíos y que el número de registros sea un entero positivo, e imprime los resultados en tablas con `tabulate`.
- **main.py**: carga los datos una sola vez, pide la consulta a la interfaz, se la pasa a la API y devuelve el resultado a la interfaz. Permite realizar varias consultas seguidas.

Nota: en el conjunto de datos la columna de la que se pide la "Topología" se llama *Topografia* (valores como Plano, Ondulado o Pendiente), por lo que es la que se muestra en la tabla.

2. Marco teórico: Diagrama de Componentes

2.1. ¿Qué es un componente?

La especificación UML 2.5.1 del Object Management Group define un componente como una parte modular de un sistema que encapsula su contenido y cuya manifestación es reemplazable dentro de su entorno [**?**]. Es decir, un componente es una unidad de software autónoma que oculta cómo está construida por dentro y se comunica con el resto del sistema únicamente a través de interfaces bien definidas [**?**]. Gracias a esto, puede sustituirse por otro componente equivalente siempre que ofrezca las mismas interfaces.

Un componente se relaciona con los demás mediante dos tipos de interfaces:

- **Interfaz proporcionada**: servicios que el componente ofrece a otros. Se dibuja como un círculo ("piruleta" o *lollipop*).
- **Interfaz requerida**: servicios que el componente necesita de otros. Se dibuja como un semicírculo ("enchufe" o *socket*).

2.2. ¿Qué es un diagrama de componentes?

Es un diagrama estructural de UML que representa, mediante una vista de diseño estática, los componentes de un sistema y las relaciones entre ellos [**?**]. Muestra cada componente como una caja negra con el estereotipo «component», las interfaces que ofrece y requiere, y las dependencias entre componentes, librerías y artefactos (archivos) [**?**, **?**].

2.3. ¿Para qué se utiliza?

- Dar una visión general de la arquitectura del sistema y documentar cómo se organizan sus partes y sus dependencias.
- Dividir un sistema en subsistemas y asignar a cada uno una responsabilidad concreta.
- Identificar qué partes se pueden reutilizar o reemplazar sin afectar al resto.
- Facilitar la comunicación entre el equipo de desarrollo y con el cliente, porque hace la estructura comprensible sin entrar en el código [**?**, **?**].

2.4. Diagrama de componentes del proyecto

La Figura 1 muestra los tres componentes del proyecto. El componente **api** proporciona la interfaz *IDatosSuelos* y el componente **ui** proporciona la interfaz *IInterfazConsola*. El componente **main** requiere ambas y es el único que conoce a los otros dos: **ui** y **api** no dependen entre sí. Esto permite, por ejemplo, cambiar la interfaz de consola por una gráfica sin modificar el cálculo de las medianas.

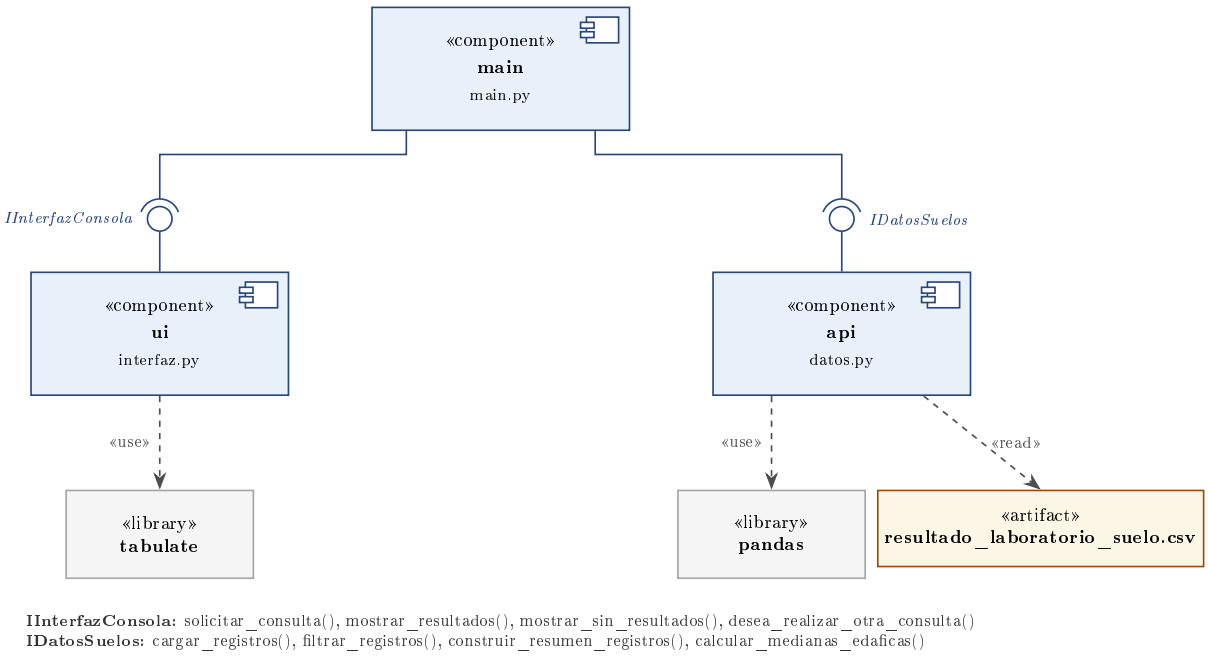


Figura 1: Diagrama de componentes de la aplicación

3. Pantallazos

Las imagenes se encuentran en el repositorio de git hub, en la seccion imagenes de prueba